

VLSI System Architecture

What does the System Architecture Team actually do?

A beginner-friendly guide for 2nd-year college students

1. The Big Picture

Every smartphone, laptop, smartwatch, car infotainment system, and game console contains at least one complex chip — usually called a System-on-Chip, or SoC. An SoC is essentially a tiny city built on a piece of silicon a few millimeters wide. It contains processors, memories, communication paths, and specialized engines for things like graphics, audio, and AI.

Designing such a chip is a huge engineering effort. It involves dozens or even hundreds of engineers, spread across many specialized teams: RTL designers, verification engineers, physical-design (layout) engineers, analog/mixed-signal designers, firmware developers, and many more.

The System Architecture team is the team that decides *what the chip should look like* before anyone starts designing it in detail. They are the chief planners.



Analogy: Think about how a building is constructed. Before a single brick is laid, an architect sits with the client and decides: how many floors, what each floor is for, where the elevators go, where the water and electricity run, how big the parking lot needs to be. Only after that plan is finalized do the civil engineers, electricians, and plumbers begin their work. The System Architecture team is exactly the same — but for a chip instead of a building.

2. Where the Architecture Team Sits in the Chip Design Flow

A chip goes through nine major stages from idea to finished product. The architecture team's work happens at the very beginning — Stage 2 in the diagram below — but the decisions made here ripple through every later stage. A bad architectural choice cannot easily be fixed in physical design or after fabrication; it has to be caught early.

The Chip Design Flow — Where the System Architecture Team Sits

The architecture team's work happens at the very beginning — and shapes everything that follows.

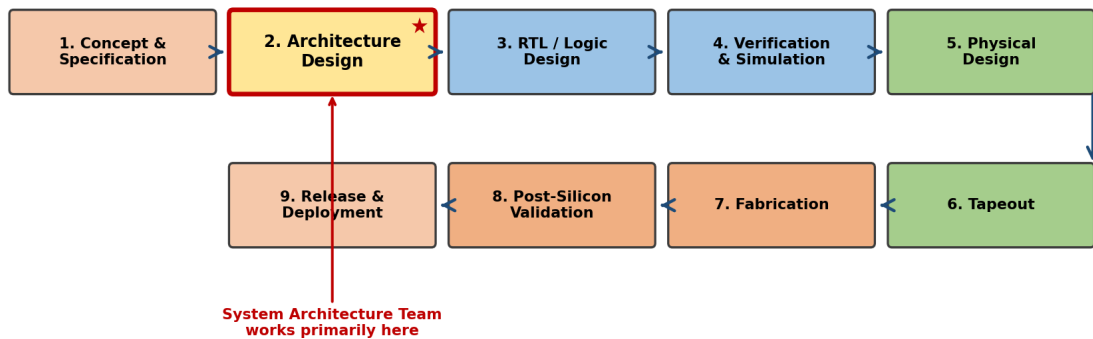


Figure 1: The chip design flow. The System Architecture team works primarily in stage 2.



In simple terms: The System Architecture team produces a written plan — called a specification — that the rest of the teams use as their blueprint. If the plan is good, the teams that follow can do their jobs efficiently. If the plan is unclear or unrealistic, every later team suffers.

3. The Seven Core Responsibilities

The architecture team has seven main responsibilities. The diagram below shows them as spokes around the team — each one is an essential part of the job.

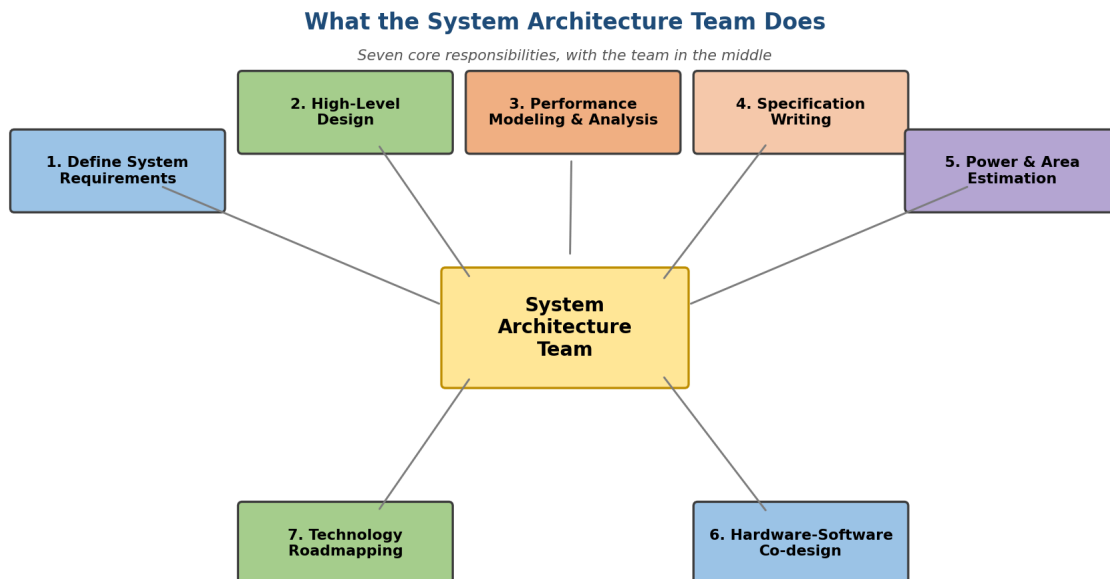


Figure 2: The seven core responsibilities of a System Architecture team.

Let's walk through each one in plain language.

3.1 Define System Requirements

Before anything else, the team must understand what the chip is actually for. Is it for a smartphone? A car? A data-center AI accelerator? A cheap IoT sensor running on a coin cell? The answer changes everything that follows.

Typical questions the team answers in this stage:

- What applications must the chip run? (mobile apps, AI inference, networking packets, etc.)
- What is the performance target? (e.g., "decode 4K video at 60 frames per second")
- What is the power budget? (e.g., "under 5 watts" for a phone, or "under 100 milliwatts" for a wearable)
- What is the area / cost target? (silicon area directly determines manufacturing cost)
- What temperature range, voltage range, and reliability requirements must it meet?

 **In simple terms:** Their job at this stage is to translate business goals ("we want a phone chip that lasts a full day") into technical numbers that engineers can actually design against.

3.2 High-Level Design

Once requirements are clear, the team decides how to organize the chip. This is where the famous "block diagram" of an SoC comes from.

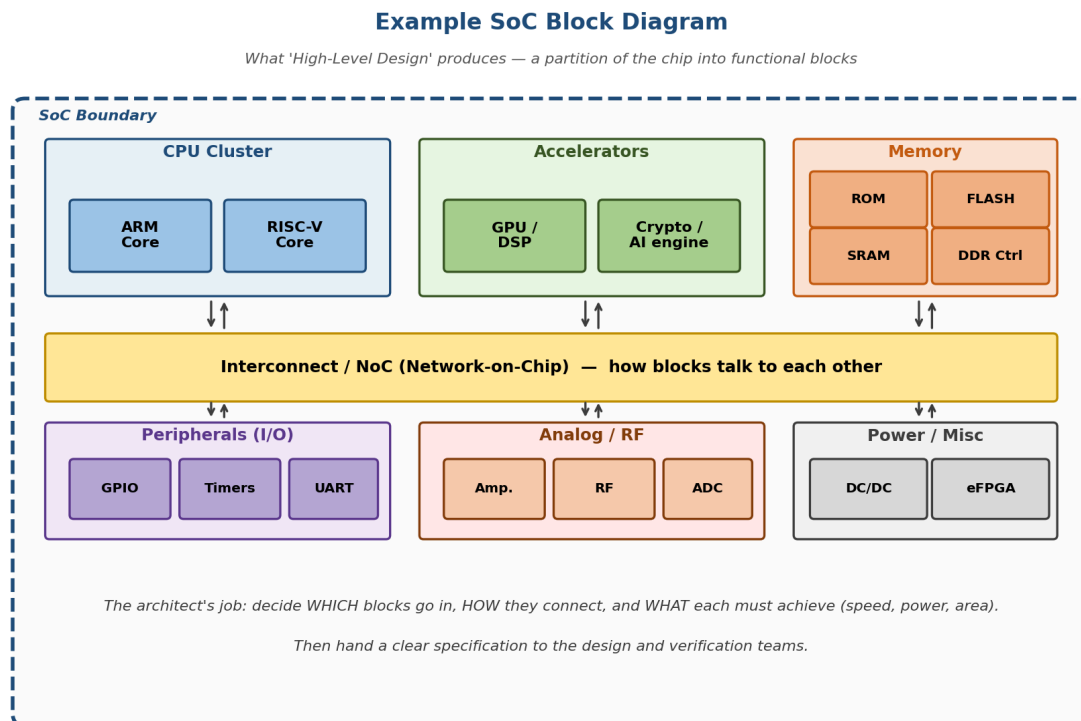


Figure 3: An example SoC block diagram — the kind of partition produced by High-Level Design.

In this stage the architecture team decides:

- Which processors to use — for example, an ARM core for general computing, a RISC-V core for control tasks, a DSP for signal processing, a GPU for graphics, an AI accelerator for neural networks.
- How much and what kind of memory — small, fast SRAM caches close to the CPU; larger but slower DRAM further away; non-volatile flash for storing programs.
- How the blocks talk to each other — through a simple bus (good for small chips) or a Network-on-Chip (NoC) for larger, more complex SoCs.
- Which functions go in dedicated hardware vs. which can be done in software — this is a major performance / area / flexibility trade-off.

 **Analogy:** If the chip is a city, this stage is about deciding the zoning plan: residential here, industrial there, schools, hospitals, and the road network connecting them. You don't yet decide the color of individual houses — that comes later when other teams handle the detailed implementation.

3.3 Performance Modeling and Analysis

Before any actual hardware is designed, the architecture team builds simplified computer models — usually in C++, SystemC, Python, or specialized simulators — to estimate how the system will behave. They are essentially building a software prototype of the chip.

With these models they answer questions like:

- If we use 4 CPU cores instead of 8, do we still meet the performance target?
- How much cache is enough? At what point does adding more cache stop helping?
- Where are the bottlenecks? Is it memory bandwidth? CPU compute? The interconnect?
- How does the system trade off power vs. performance vs. area? ("PPA")

 **In simple terms:** PPA — Power, Performance, Area — is the holy trinity of chip design. Almost every architectural decision is a trade-off among these three. More performance usually costs more area and more power. The architect's skill is finding the sweet spot for the target application.

3.4 Specification Writing

Once the high-level decisions are made, the team writes them down as detailed specifications — the documents the rest of the company will use to actually build the chip.

Typical things specified:

- Instruction sets — exactly which instructions a custom processor must support, with their bit encodings.
- Protocols — the precise rules for how blocks communicate (e.g., "the AI accelerator gets new work using these handshake signals, and reports completion in this register").
- Memory maps — which address ranges correspond to which block. (e.g., 0x0000_0000 – 0xFFFF_FFFF is DRAM, 0x4000_0000 onward is the GPU's control registers, etc.)
- Configuration registers — the software-visible "knobs" that firmware can use to change the chip's behavior at runtime.



Analogy: If high-level design is the floor plan, specification writing is the construction blueprint that says exactly where each window goes, what size each door is, and which way the plumbing runs. Builders follow it precisely.

3.5 Power and Area Estimation

Long before the chip is laid out, the architecture team must give realistic estimates of how much silicon area each block will need and how much power it will consume. These estimates feed back into the architectural choices and help the company decide whether the design is even feasible at the target cost and battery life.

They use a mix of techniques: scaling data from previous chip generations, lookup tables from the silicon foundry (TSMC, Samsung, Intel Foundry, GlobalFoundries, etc.), early synthesis runs of similar blocks, and analytical models. They work hand-in-hand with the design and verification teams to keep these estimates honest as the project progresses.



In simple terms: If the area or power estimate looks bad, the architecture team must go back and rethink the design — remove a feature, simplify a block, or change the target technology node. Catching these problems early is exactly why this stage exists.

3.6 Hardware-Software Co-design

Modern SoCs are useless without software. Operating systems, drivers, firmware, libraries, and applications all need to run on the chip. The architecture team works closely with the firmware and software teams to make sure the hardware they design is actually programmable and pleasant to use.

This involves things like:

- Making sure each hardware block exposes a clean, sensible set of registers that drivers can use.
- Choosing whether a feature should be done by dedicated hardware (faster, less flexible) or by software running on a CPU (slower, but easier to fix and update).
- Defining the interrupt structure — how the hardware tells the CPU "I'm done" or "I have new data".
- Ensuring there is a clean boot flow, debug support (JTAG, trace), and security model (secure boot, key storage).



Analogy: A great kitchen is no good if the chef hates working in it. The architecture team makes sure the hardware is one that software engineers want to write code for — predictable, well-documented, and with no nasty surprises.

3.7 Technology Roadmapping

Finally, while one chip is being built, the architecture team is already thinking about the next one — and the one after that. They evaluate emerging technologies and decide what to bet on for future products.

Examples of things they evaluate:

- New process nodes (3 nm, 2 nm, ...) — what they cost, when they'll be ready, what they enable.

- Chiplets and advanced packaging — splitting a large chip into smaller "chiplets" that are stitched together in a package.
- 3D stacking — placing memory directly on top of compute logic to dramatically increase bandwidth.
- New IP blocks — emerging accelerators for AI, security, networking, etc.

 **In simple terms:** Chip projects take 2-4 years. The architecture team is always working at least one project ahead, so the company is never caught flat-footed when a new technology becomes mainstream.

4. What Skills Does a System Architect Need?

Because the role spans so many areas, system architects tend to be experienced engineers (often 8+ years in the industry) who have already worked in one or more specialized teams — RTL design, verification, performance modeling, or firmware — before moving into architecture. Common skills include:

- Solid grasp of computer architecture: pipelines, caches, memory hierarchies, coherence, virtual memory.
- Understanding of digital design and HDLs (Verilog/SystemVerilog) — enough to read RTL and reason about feasibility, even if they're not the ones writing it day-to-day.
- Programming and modeling skills: C/C++, Python, sometimes SystemC, for building performance models and prototyping.
- Awareness of physical realities: how transistor count translates to area, what limits clock frequency, what burns power.
- Knowledge of standard interfaces and protocols: AXI, AHB, PCIe, DDR, USB, Ethernet, etc.
- Strong communication and writing skills — half the job is producing clear specifications and convincing other teams of design choices.
- Trade-off thinking — being comfortable with the fact that there is rarely a single "correct" answer, only better and worse compromises.

5. What Does a Typical Day Look Like?

Architects spend much less time writing code or HDL than designers do. A typical day mixes:

- Meetings with product management to clarify requirements.
- Discussions with design and verification teams to resolve ambiguities in the spec.
- Running performance simulations and analyzing the results in spreadsheets / plots.
- Writing or revising specification documents.
- Reviewing competitor products, research papers, and roadmaps from IP vendors and foundries.
- Sketching block diagrams, drawing flow charts, debating trade-offs on whiteboards.

6. How Can a Student Prepare for This Career?

If becoming a chip architect interests you, here are concrete things you can do as a 2nd-year student to start moving in that direction:

- **Master the fundamentals first:** digital logic, computer organization, basic Verilog or VHDL, and one assembly language (RISC-V is excellent because the spec is free and approachable).
- **Take a computer-architecture course:** pipelines, caches, branch prediction, virtual memory, multicore. Patterson & Hennessy is the classic textbook.
- **Build something:** design a small RISC-V CPU on an FPGA, or implement a simple cache simulator in C++ or Python. Hands-on projects beat theory alone.
- **Learn to read research papers:** ISCA, MICRO, HPCA and ASPLOS are the big computer-architecture conferences. Many papers are free to read online.
- **Get comfortable with PPA thinking:** every time you build something, ask: "What does this cost? What does this gain? What's the trade-off?"
- **Communicate clearly:** write a one-page design doc for every project you do. Architects live by their documents.

7. Quick Glossary

SoC (System-on-Chip) — A single chip that contains an entire computer system: processors, memory, peripherals, and interconnect, all on one piece of silicon.

IP block — Intellectual Property block — a pre-designed, reusable piece of a chip (e.g., a USB controller, an ARM core) that companies license and integrate into their SoCs.

RTL (Register-Transfer Level) — The level of abstraction at which digital designers describe hardware behavior in HDLs like Verilog or VHDL — what data moves between registers on each clock cycle.

NoC (Network-on-Chip) — A communication fabric inside a chip that routes data between blocks like a tiny computer network, used in large SoCs instead of a simple shared bus.

PPA — Power, Performance, Area — the three primary metrics chip designers optimize and trade off.

Tapeout — The moment the final chip design layout is sent to the foundry for manufacturing. Mistakes after tapeout are extremely expensive.

Process node — A generation of manufacturing technology, named by feature size (e.g., "5 nm", "3 nm"). Smaller nodes pack more transistors but cost more to develop.

Chiplet — A small chip designed to be combined with other chiplets in a single package — letting designers mix and match smaller dies instead of building one huge monolithic chip.

Specification ("spec") — The detailed document the architecture team writes to tell the rest of the engineering organization exactly what to build.

*The System Architecture team is the bridge between the product idea and the silicon that makes it real.
Good architecture makes everyone else's job easier. Bad architecture is felt for years.*